Of course, you might find yourself needing to encrypt data manually; not every PHP system out there is running via Apache or Nginx. It's also a fun exercise! So let's have a go and roll our own encryption code. PHP offers two modules we can use for this task: `openssl` and `sodium`. Most installations tend to offer both by default. Both extensions are very capable and offer tools for key/certificate management, hashing, side-channel attack prevention (preventing the inference of information from timing and memory use), encoding, and of course encryption. We'll use `sodium` here, but that's purely personal preference on my part, and two methods: using a shared secret and using a public/private key pair.

Shared Secret (Symmetric)

In time-honored tradition, we have two secret agents: Alice and Bob. Alice and Bob have a previously agreed-upon secret key, which they will both use to encrypt and decrypt messages to each other. We can use sodium's `sodium_crypto_secretbox()` function to encrypt messages, which takes three arguments: the message to be encrypted, a one-time use random byte sequence (called a *nonce*, which has an unfortunate meaning in modern British slang but is an old Middle English word for something temporary), and the shared secret. The shared secret has to be a certain number of bytes, and we can generate them with the `sodium_crypto_secretbox_keygen()` function:

```
$secret = sodium_crypto_secretbox_keygen();;
$message = 'WE RIDE AT DAWN.';

$nonce = random_bytes(SODIUM_CRYPTO_SECRETBOX_NONCEBYTES);
$encrypted_message = sodium_crypto_secretbox($message,
    $nonce,
    $secret
);

print bin2hex($message) . PHP_EOL;
print bin2hex($encrypted_message) . PHP_EOL;

Output:
57452052...
11e4274e...
```